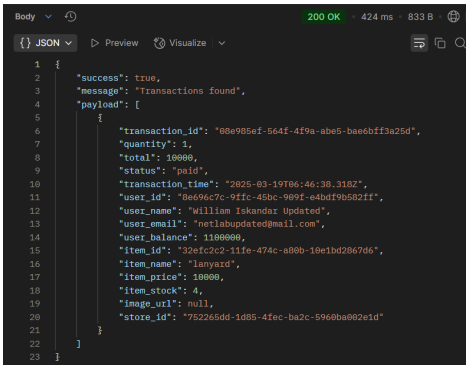
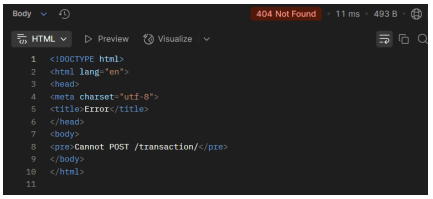
PRAKTIKUM
SISTEM BASIS DATA

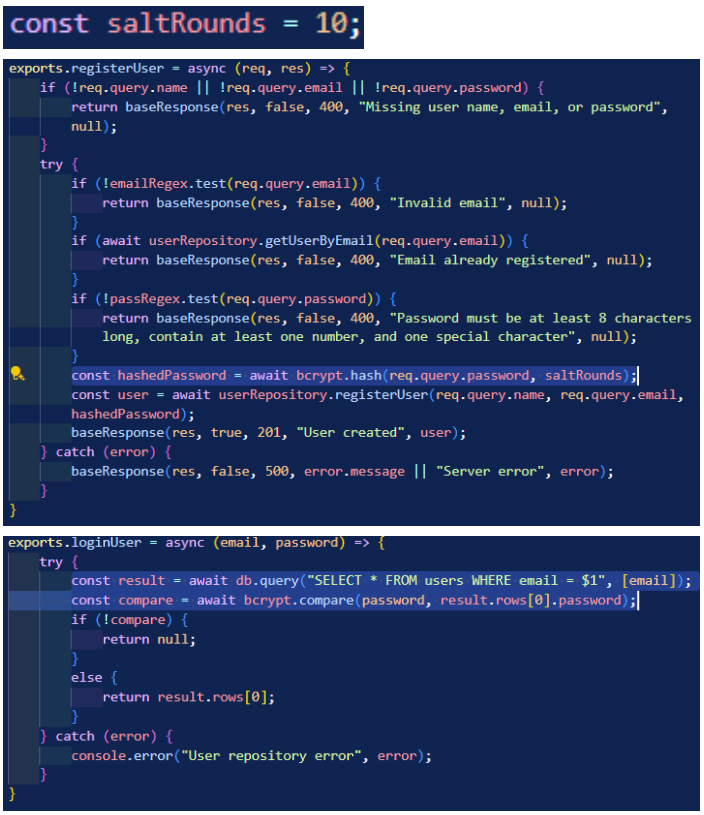
Nama	**Jonathan Frederick Kosasih**	**No. Modul**	**6**
NPM	**2306225981**	**Tipe**	**TUTAM**

1. Saya menambahkan fungsi getAllTransactions()
2. Screenshot:

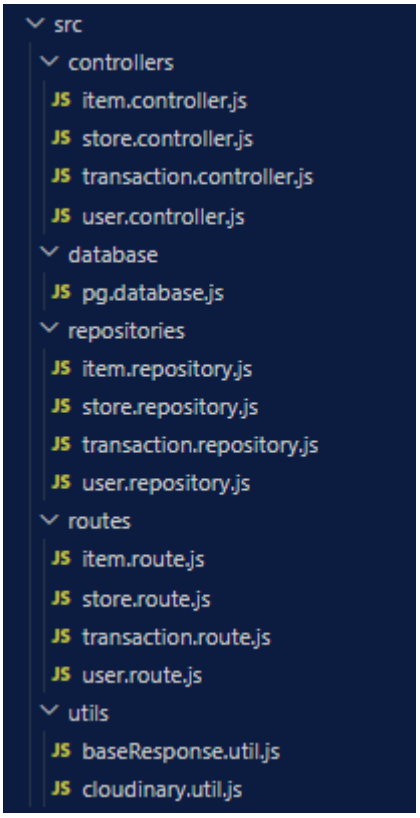
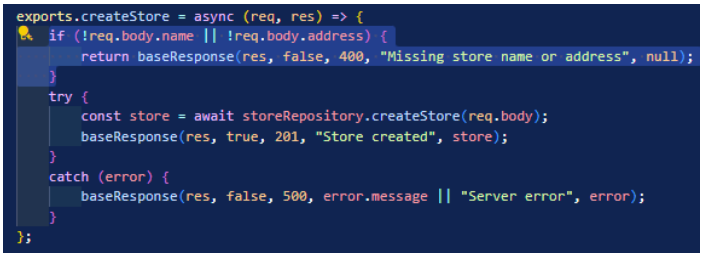
Method	Endpoint	Success	Failed
GET	/transaction		

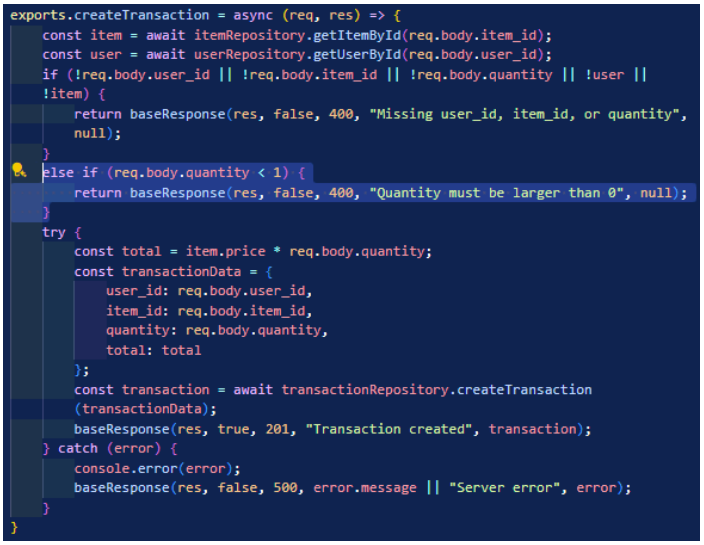
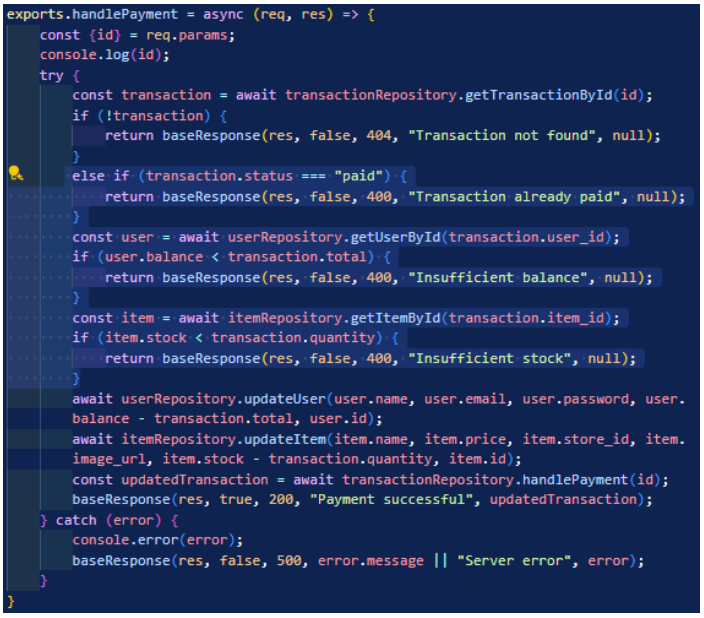
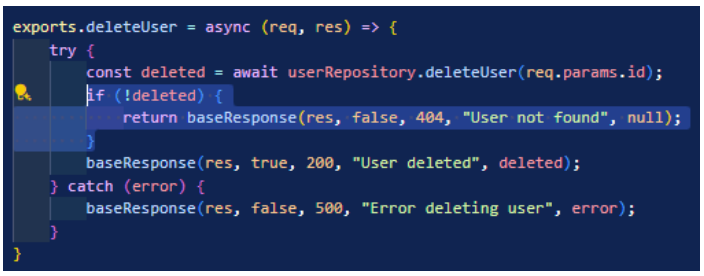
3. Penjelasan fitur untuk meningkatkan scalability, security, dan error handling:

Tipe	Implementasi	Deskripsi
Scalability		Pengecekan email menggunakan emailRegex saat register user dan update user, dikarenakan kemungkinan terjadinya penggunaan email dengan bentuk yang tidak valid untuk memudahkan pengolahan data

Security		Pengecekan password menggunakan passRegex saat register user dan update user, dikarenakan kemungkinan terjadinya penggunaan password yang terlalu mudah untuk dibobol.
Security		Penerapan hashing dengan metode bcrypt pada password agar password dapat tersimpan dalam bentuk terenkripsi dan hanya dapat didekripsi kembali menggunakan compare saat login user.
Security		Penggunaan middleware CORS untuk mengontrol asal request agar hanya bentuk request tertentu dari domain asal tertentu yang dapat masuk ke dalam program.

Scalability	src > utils > JS cloudinary.util.js > ... 1 require("dotenv").config(); 2 const cloudinary = require("cloudinary").v2; 3 4 cloudinary.config({ 5 cloud_name: process.env.CLOUDINARY_CLOUD_NAME, 6 api_key: process.env.CLOUDINARY_API_KEY, 7 api_secret: process.env.CLOUDINARY_API_SECRET 8 }); 9 10 const uploadImage = async (image, folder) => { 11 return new Promise((resolve, reject) => { 12 if (!image) { 13 return resolve(null); 14 } 15 const base64 = Buffer.from(image.buffer).toString("base64"); 16 const url = `data:${image.mimetype};base64,${base64}`; 17 cloudinary.uploader.upload(url, { folder }, (error, result) => { 18 if (error) { 19 return reject(error); 20 } 21 else { 22 return resolve(result.secure_url); 23 } 24 }); 25 }); 26 } 27 28 module.exports = { 29 uploadImage, 30 ...cloudinary 31 }; exports.createItem = async (req, res) => { const store = await storeRepository.getStoreById(req.body.store_id); if (!req.body.name || !req.body.price || !req.body.store_id || !req.file || !req.body.stock) { return baseResponse(res, false, 400, "Missing item name, price, store_id, image, or stock", null); } else if (!store) { return baseResponse(res, false, 404, "Store doesn't exist", null); } try { let image_url = null; if (req.file) { image_url = await cloudinary.uploadImage(req.file, "items"); } const itemData = { name: req.body.name, price: req.body.price, store_id: req.body.store_id, stock: req.body.stock, image_url: image_url }; const item = await itemRepository.createItem(itemData); baseResponse(res, true, 201, "Item created", item); } catch (error) { console.log(error); baseResponse(res, false, 500, error.message || "Server error", error); } }	Integrasi dengan API Cloudinary untuk mengunggah gambar untuk item yang dibuat dan menyimpannya dalam bentuk URL ke dalam database.

Scalability		Program dipisahkan menjadi banyak bagian kecil-kecil (repository, controller, route, dan util) untuk memudahkan ekspansi dan troubleshooting pada program.
Error Handling		Memastikan body tidak kosong saat pembuatan atau pembaruan data agar tidak adanya data krusial yang bernilai NULL pada database.
Error Handling		Memastikan bahwa nilai balance pada user tidak bernilai negatif.

Error Handling		Memastikan jumlah barang yang ingin dibeli bukan merupakan bilangan negatif sebelum membuat transaksi.
Error Handling		Memastikan transaksi belum terbayarkan, nominal mencukupi, dan stok barang mencukupi sebelum melakukan pembayaran.
Error Handling		Memastikan bahwa objek yang terlibat ada sebelum diperbarui atau dihapus.